NAME(S):

Begin the following as paired-programming in class, but be sure to individually finish any problems you don't get to in order to be prepared for the practical exam

1. St. Nicholas' house

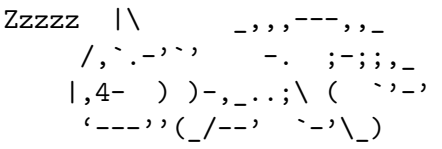
1. Download the starter code `house.py` from the course website. The file contains code to draw a house-like shape .
    - German kids know this shape as "St. Nicholas' house". It can be used as a puzzle where you have to draw the shape without lifting your pencil and without tracing.
2. Define a function called `house` that draws the shape.
3. Define a function `fly_to` that does the same things as the `turtle.setpos` / `turtle.goto`, but doesn't leave a line behind as the turtle moves.
4. Use these functions to draw a few houses in different locations
5. Modify the `house` function so that it allows you to draw houses of different sizes and colors. Start by figuring out how many parameters your function needs and what each parameter represents.
6. Draw a picture of houses in different colors and sizes. For example:

Figure 1: St. Nick's Houses

7. Modify your program so that the size and location of the houses is random.

2. ASCII art

ASCII art uses keyboard characters to create pictures. For example:



1. Define a function called ascii_tree that uses the print function to display the following picture:

```
   *
  ***
 *****
*******
  ***
```

2. Now add two parameters to the function which specify which characters to use for the branches and the trunk. The function should now be able to print the following:

```
   o
  ooo
 ooooo
ooooooo
  nnn

   -
  ---
 -----
-------
  ###

   ~
  ~~~
 ~~~~~
~~~~~~~
  vvv

   i
  iii
 iiiii
iiiiiii
  uuu
```

3. Function parameters vs. user input

Consider the following two programs:

```
1)  # Temperature Conversion Version 1
2)  def fahrenheit_to_celsius(fahrenheit):
3)       celsius = (fahrenheit - 32) * 5 / 9
4)       print(fahrenheit, "deg Fahrenheit is", end=" ")
5)       print(celsius, "deg Celsius.")
6)
7)  temperature = int(input("What is the temperature in Fahrenheit? "))
8)  fahrenheit_to_celsius(temperature)
```

```
1.  # Temperature Conversion Version 2
2.  def fahrenheit_to_celsius():
3.       fahrenheit = int(input("What is the temperature in Fahrenheit? "))
4.       celsius = (fahrenheit - 32) * 5 / 9
5.       print(fahrenheit, "deg Fahrenheit is", end=" ")
6.       print(celsius, "deg Celsius.")
7.
8.  fahrenheit_to_celsius()
```

1. From the user's point of view, do these programs behave differently?
2. From the programmer's point of view, which one is better and why?

    - Save your answers as comments in a `.py` file, which you'll use for the next problem.

4. Bookstore

Define a function called `book_order`, which should take the cover price of a book and the number of copies the store wants to order as its parameters. Suppose that bookstores get a 40% discount. Shipping costs $3 for the first copy and $0.75 for each additional copy. The function should print out a message indicating

 the cover price of the bok the number of copies ordered what the bookstore will have to pay for the books (don't forget the discount) the shipping cost the total amount the bookstore will have to pay (for books + shipping)

For example, if you called `book_order(12.97, 15)` (meaning the store orders 15 copies of a book that costs $12.97), it should print:

```
cover price: $12.97
copies ordered: 15
order cost: $116.73
shipping: $13.50
total: $130.23
```

Hint: you can use the built-in round function to `round(<value>, <decimal places>)` to a desired number of digits after (or before) the decimal point. You can find the documentation in the Python Library Reference section on Built-in Functions, which can also be found through the Resources section on the course website: https://docs.python.org/3/library/functions.html

Don't forget to comment your code and organize it in a sensible way.

Don't forget to include your answers from section 3 as comments in the code.

Save your program in a file like `bookstore.py`.

User input and output vs. function input and output

- user input/output: to communicate with the *user* of a program.
  - **user input**: built-in function `input()`
  - **user output**: built-in function `print()`
- function input/output: for different parts of a program to communicate with each other. e.g. between two functions, between the global namespace (scope) and a function's namespace (scope)
  - **function input**: the function's parameters
  - **function output**: the function's return value

Functions that take parameters and return values are more versatile than ones that directly ask a user for input and respond accordingly. Functions that take parameters and return results can still be used in programs that ask for user input then call the function with the user input as parameter values.

In contrast to functions that take parameters and return values, functions that rely on direct user input are more limited in their applicability. Specifically, these functions can't be used as building blocks for larger programs where the function needs to be able to handle values calculated by other parts of the program or where the function's results need to be processed further.

Please be sure to read problem specifications carefully. Unless the instructions specify otherwise, generally, you should assume that any input values the function requires should be provided as parameters and that any results the function calculates should be returned.

1. Improve the temperature conversion function from last time by re-writing it to take a temperature in degrees Fahrenheit as a parameter, then **return** the temperature in degrees Celsius.

6 Defining functions with return values

1. Circle area

Define a function called `circle_area` that calculates the area of a circle. Given the circle's radius, the function should return the area.

The area of a circle is given by the following equation:

$$Area = \pi r^2$$

Python's math module contains a variable called pi that gives you an approximation of π. Given what you know about modules, how can you use this predefined variable in your code?

Once you have imported the math module (`import math`), you can access π with: `math.pi`

- Note the lack of parentheses. Pi, here, is a variable, not a function, so no parentheses are used.

2. Pizza cost per square inch

Define a function called `pizza_cost` that calculates then returns the cost per square inch for a circular pizza, given the pizza's diameter and total price. You should use the `circle_area` function from the previous question to help calculate the expected value.

3. Coffee intake

Define a function that calculates how much coffee will fit into a cylindrical mug, given its radius and height. You should be able to reuse the `circle_area` function you've previously defined.

The volume of a cylinder is given by:

$$V_{cylinder} = A_{circle} * height$$

4. Random Sizes

Test your functions by using the `random` module to generate random sizes for the radius of a circle, cost of a pizza, and height of a mug.

Prompt: Is randomly generating values creative? Does it produce novel information? Is it "just" following instructions? Why or why not?